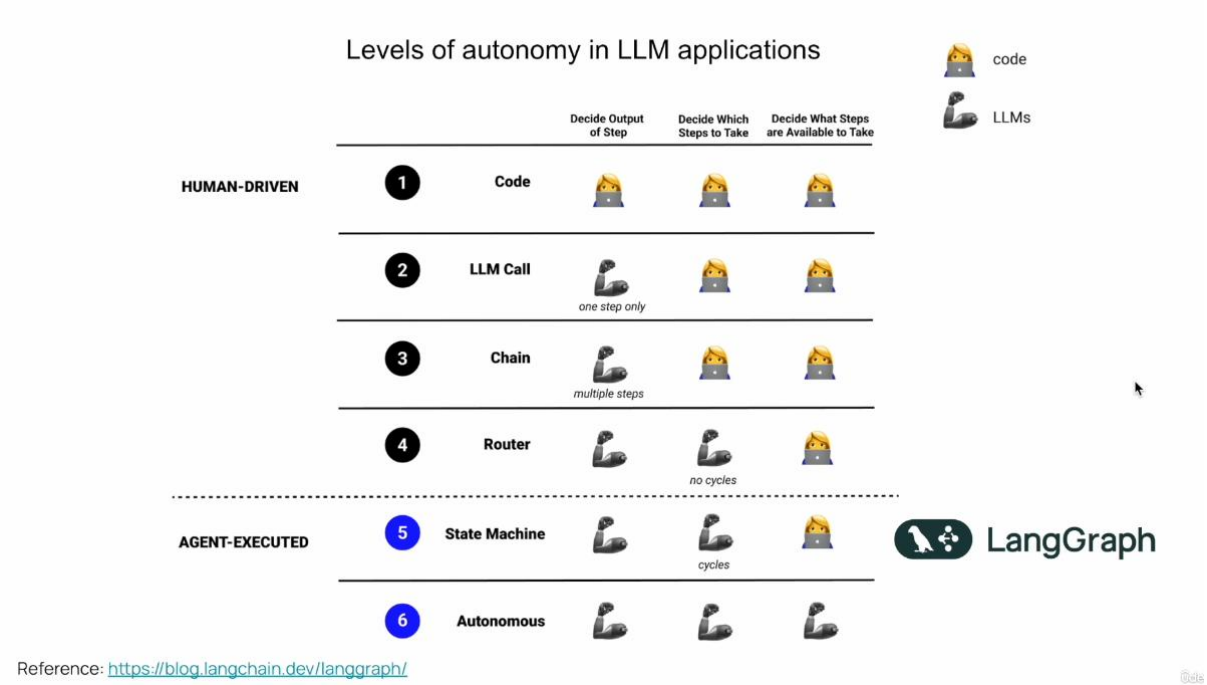
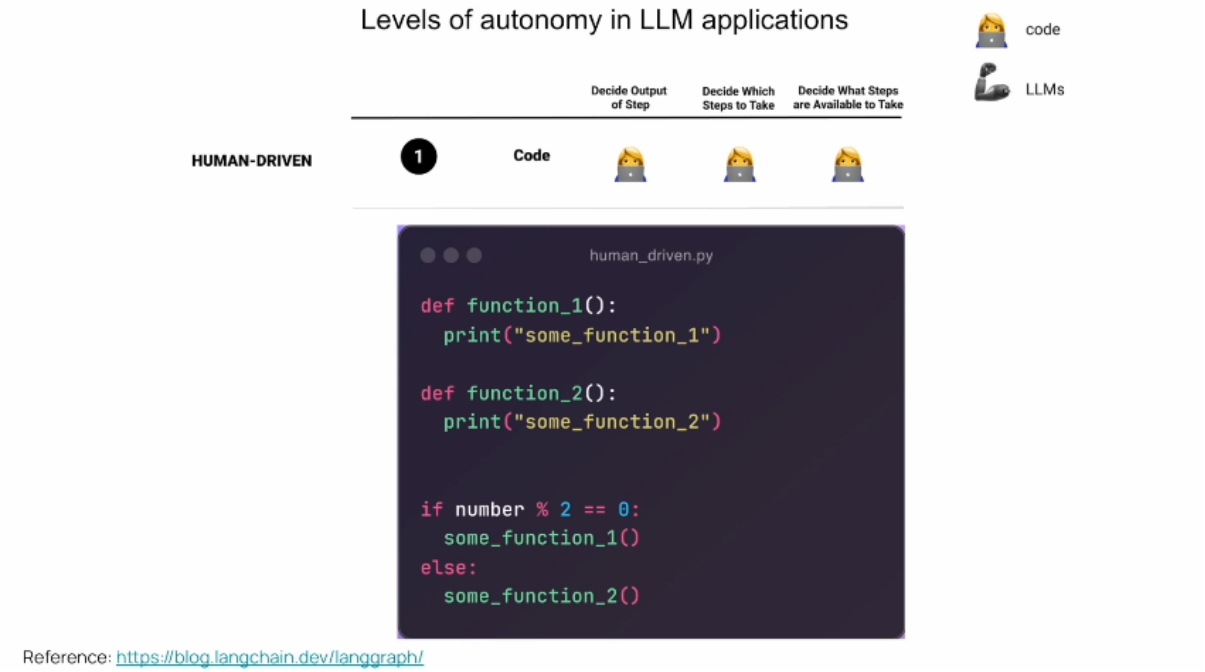
Levels of Autonomy in LLM Application



Reference: https://blog.langchain.dev/langgraph/

Level 1:



Reference: https://blog.langchain.dev/langgraph/

if we'll take a look on AI systems from the perspective of levels of autonomy that those applications have, then we have a spectrum.

And in that spectrum on the one end we have deterministic code.

And those are systems where we as developers we write the code.

We do not integrate any LLM.

So we only have deterministic code like the functions and flow that you see right over here.

So we know exactly what is the output which is going to be at every step, what's going to be the input,

which steps we're going to take.

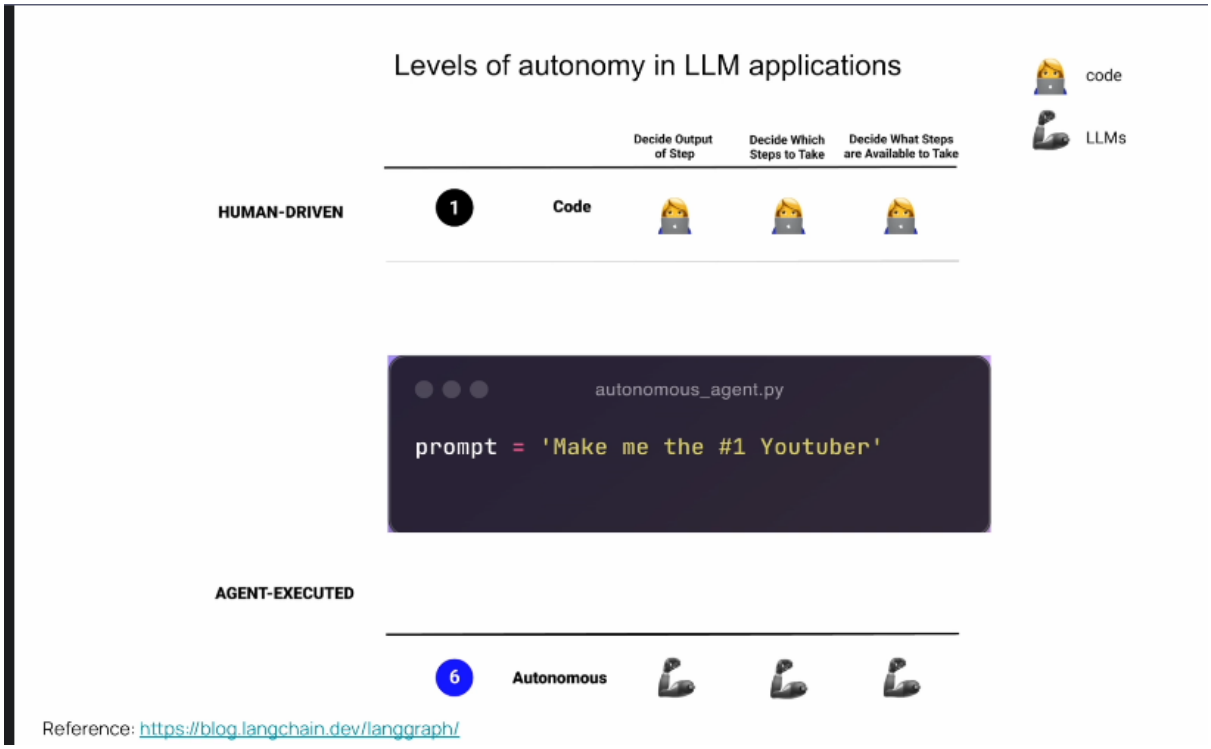
And we basically have control on the entire system.

And while those systems are very resilient and reliable because we have control on their entire performance.

However, they're not flexible at all.

And this is because everything is hard coded.

Level 6:



And if we take a look at the spectrum, on the other hand, we have this idea of autonomous agents.

And those autonomous agents can do everything.

They can make up their tasks of what to do.

They can write code, execute that code, then to reorder their tasks and to write another code. And they're very dynamic, and they can really get a task from the beginning to the end.And they are super flexible.And they can receive prompts like, **make me the number one YouTuber**.

And they're supposedly can actually do this.

However, in reality, those kinds of systems, they don't really exist.

So projects like auto, GPT, GPT, engineer, Baby AGI, they were trying to to implement something

like this.

Now, while those projects are very important to the industry, and I really think they pound on innovation and pushing the boundaries, they're not very production oriented, and we don't see any usage of those kinds of systems in production because they're simply too flexible.

And we don't have control because we rely too much on the LLM.

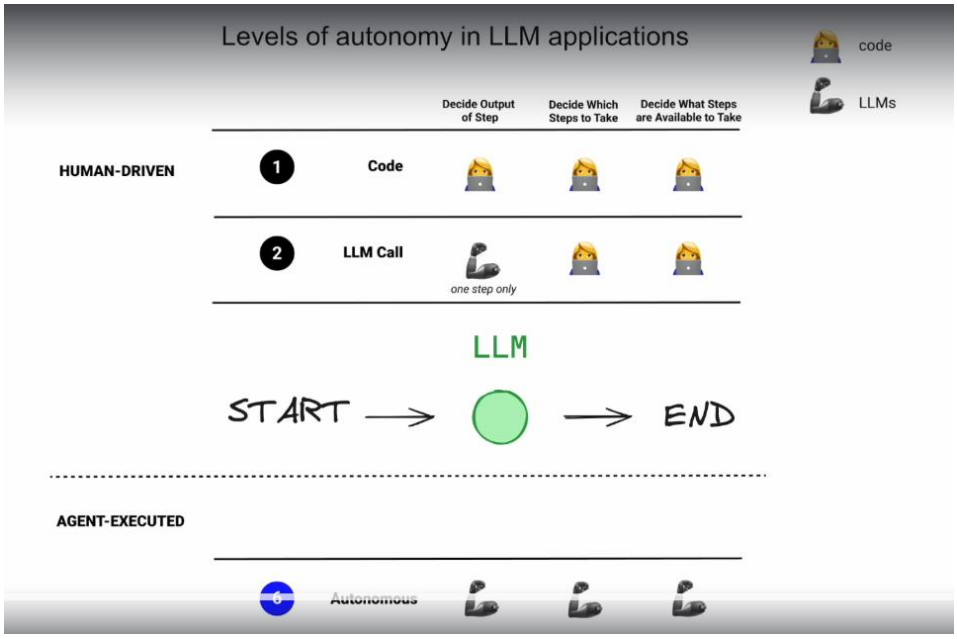
And when we do that, then the LLM tends to scatter around and to not output us what we want.

And the reason for this is that llms, at the very basic level, are simply statistical creatures,

guessing one token after the other. So autonomous agents, while flexible, they're not very reliable.

So those are the two ends of the spectrum.

Level 2:



Let's now talk about what's in between this spectrum.

So one level after writing deterministic code is to integrate an LLM into that code.

So we as developers we still write the code and we control the control flow.

And we know exactly what's going to be executed.

But inside of this flow then the LLM can be used for example, maybe to summarize something or to extract

information to extract some entities.

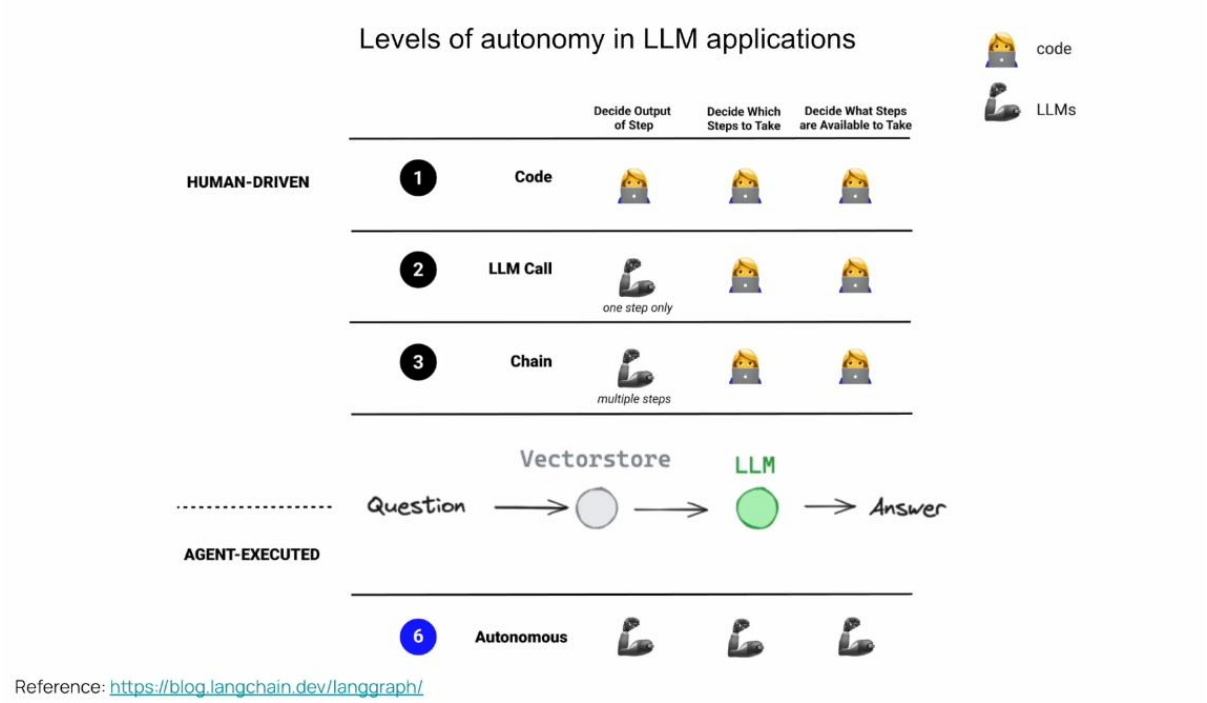
But overall we have a lot of control over here.

The LLM simply helps us in this process and controls only one output in this entire flow.

And by only doing this, by integrating an LLM, we gain a lot of flexibility because the generation

of llms can be very creative, and we still have most of the control on the development side.

Level 3:



Reference: https://blog.langchain.dev/langgraph/

Now, if we take this a step further, we can introduce the concept of chaining.

And this is basically to take one output of LM and giving it as an input to another LM.

So to compose one call over the other.

And here you can see an example of a rack flow retrieval augmentation generation where we give the first

LM our original question.

We then integrate embeddings.

So we take the question, we embed it and we retrieve with similar relevant documents which are probably

going to help answer that question.

And then we take the original prompt.

We augment it and we send everything to the LM which finally generates the answer.

And this is just one example of chaining multiple LM calls using embeddings.

So we have a lot of other use cases and a lot of other cool chains.

But this is basically one step further, which really gives us the possibility to build really cool

things.

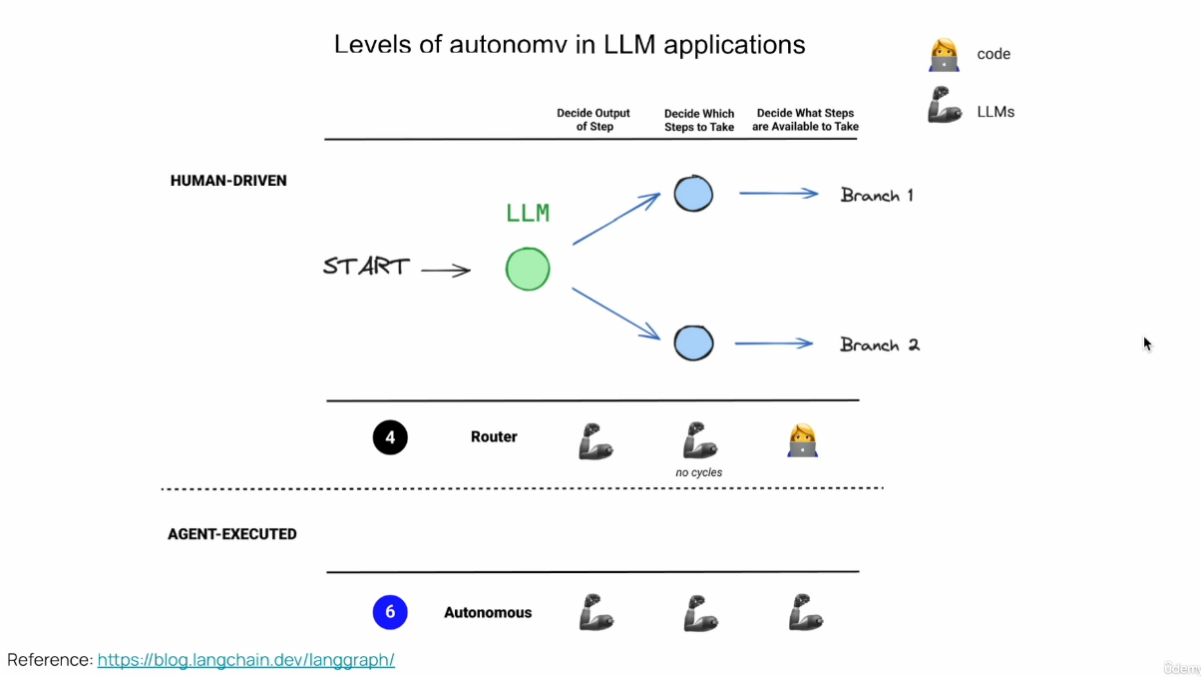
So to summarize, in a chain, we have llms that determine the output in a bunch of steps, not only

one.

And the more we progress to this autonomous agent end of the spectrum, we leverage Llms to build complex

systems.

Level 4:



Reference: https://blog.langchain.dev/langgraph/

let's now discuss the concept of an LLM router.

An LLM router is a kind of chain which leverages the LLM to decide where do we want to go.

We can use an LLM to decide whether we're going to execute code in branch one or coding branch two.

So it can be for example to go and search in a database or to go and search over the web.

And the LLM can decide where to go.

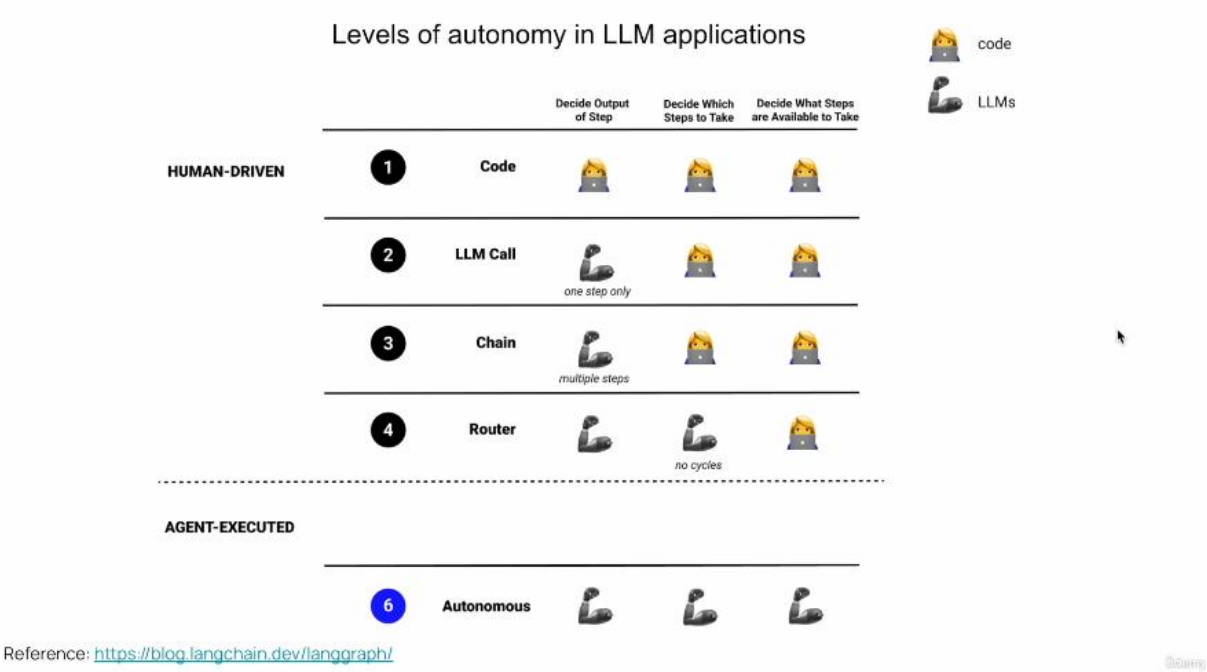
So we're using the reasoning power of an LLM here.

Here for the first time the LLM decides which steps do we need to take?

And this gives us even more flexibility of building those kinds of systems.

However, you notice here that it's written that there are no cycles in an LLM router.

And you see this dotted line over here.



So what is below this dotted line is what we consider an agent or an agentic application.

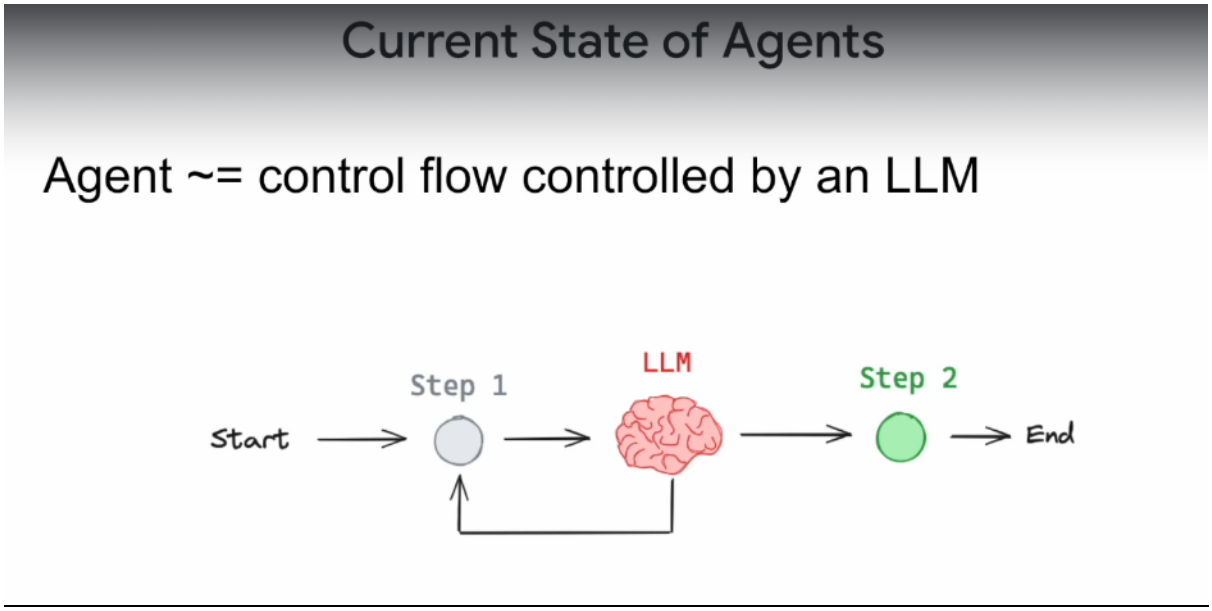
So everything you see above this line is actually very well implemented within LangChain.

We can build very advanced systems with only those building blocks with the LangChain framework.

You see this gap here right between this autonomous agent and the router.

An agent is essentially a control flow that an LLM decides where to go.

So you can see in this picture over here we have an LLM which decides whether to go to step one or whether to go to step two.



So this is a very basic example of an agent.

We're using the reasoning power of an LLM to decide where to go in this flow.

You might ask yourself, hey, what's the difference between an agent and a chain and specifically even the router chain?

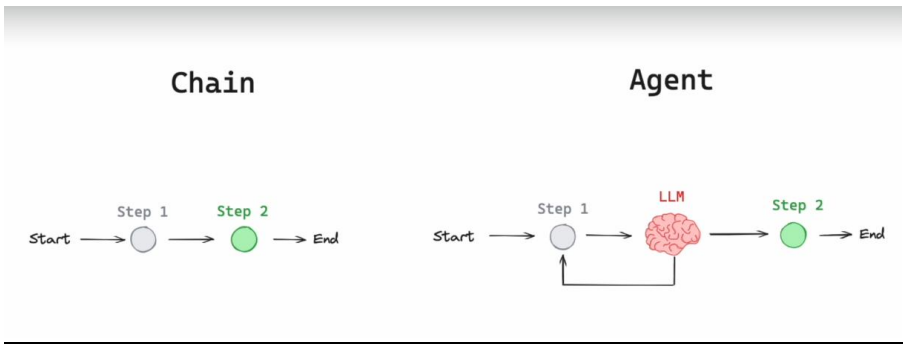
And the main difference is that a chain is one directional.

We move from the left to the right while in an agent we actually have those kinds of cycles.

And those cycles we're going to soon see are very important.

And they are what's going to give our application a genetic properties.

And specifically agents, at least agents.



Nowadays they use function calling to decide which steps to take.

And function calling is a very cool feature of certain llms where Besides our query that we sent the

LLM the question, we can send a description of tools.

So those are functions that we can execute in our backends and we can orchestrate them.

And we send the LLM also the description of those functions, their arguments, their title, what they're

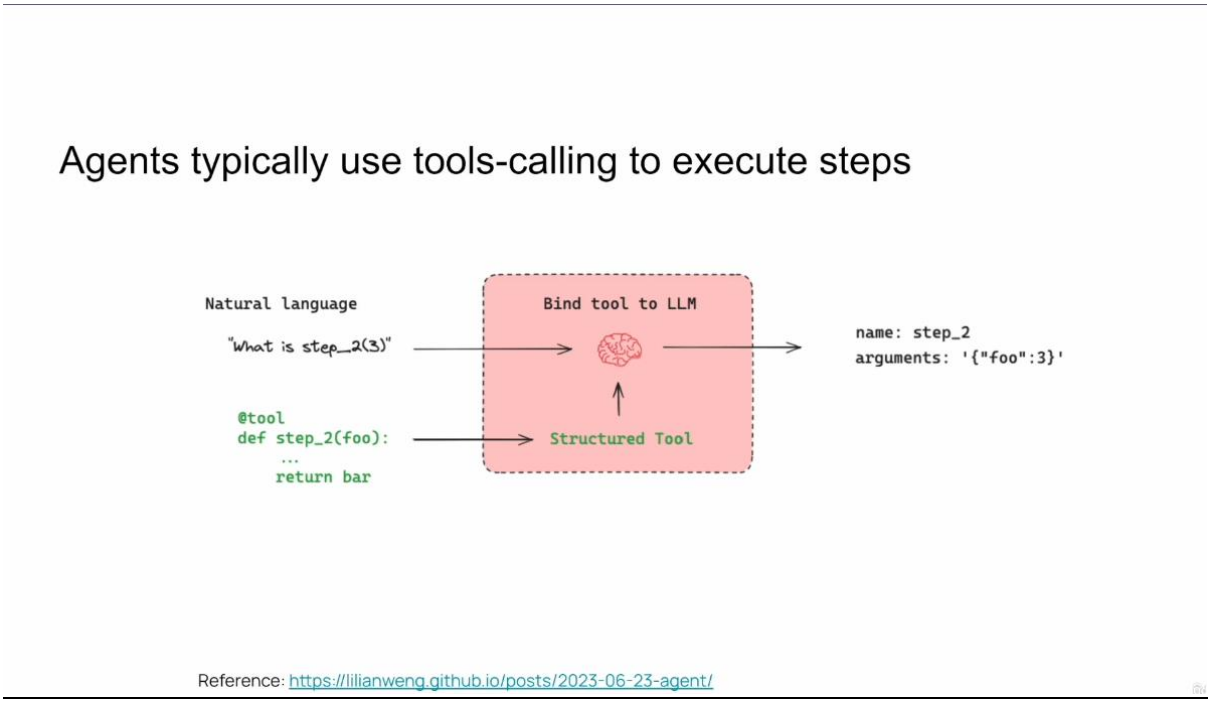
supposed to do and what their return value is.

And we can do that very easily with the tool decorator and then the LLM, if it finds appropriate,

it can tell us that we need to invoke these functions with those arguments.

So hopefully we'll go will invoke those function with those arguments.

And we'll get back the answer that we want.



---

now let's talk about the most basic design for an agent.

And it was first introduced in the react paper.

And it really I think changed our industry.

So basically the algorithm for this type of agent is very simple.

So we start.

We then use the LM to decide whether we need to use a tool.

So for example, to make a call to an API or a call to a database and query there and decides whether we need to call this tool or not, then we go and call this tool with the arguments that the LM chose us to execute this tool, we get an answer and then we feed back everything to the LM, which can decide whether to use another tool or whether to return the answer to the user.

And it has all the permutations that it can run.

However, it was too flexible and since every permutation is allowed, then also this permutation is allowed.

And if you've been developing agents for a while, then you're probably familiar with this error where the agent is in this sort of endless loop, where it's invoking the same tool over and over and over and simply is stuck.

And there are many reasons for why this can happen.

Maybe we didn't define our tools correctly.

The LLM is non-deterministic mistake or not strong enough.

It chooses the correct tool to use, or it gives it the incorrect arguments, or it even hallucinates a tool that does not exist.

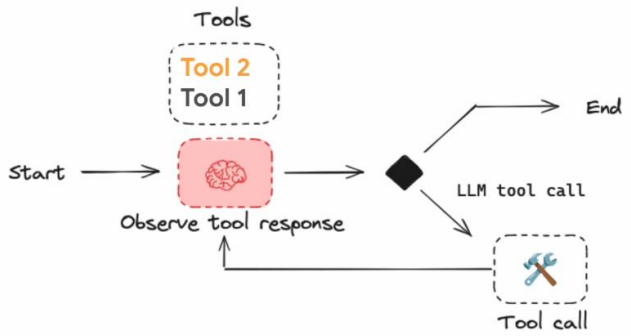
So there are a lot of explanations of why those things may happen.

And basically what I'm trying to show you here is that we have here a problem where we have an agent maybe that is flexible, like in other GPT or even the react algorithm, but it's not very reliable.

And what we want is something which still is flexible but much more reliable.

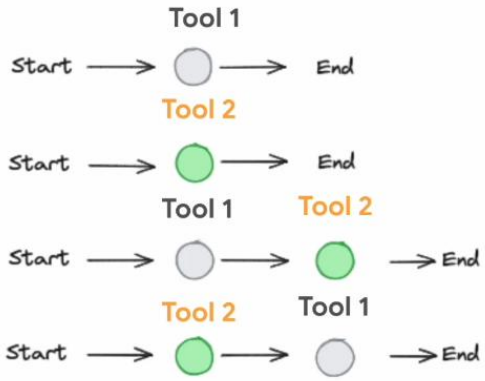
So we can actually use it in production systems and let users interact with it and get good results, which work outside the scope of a demo.

Simplest design is a `for loop` (ReAct)



Reference: https://react-lm.github.io/

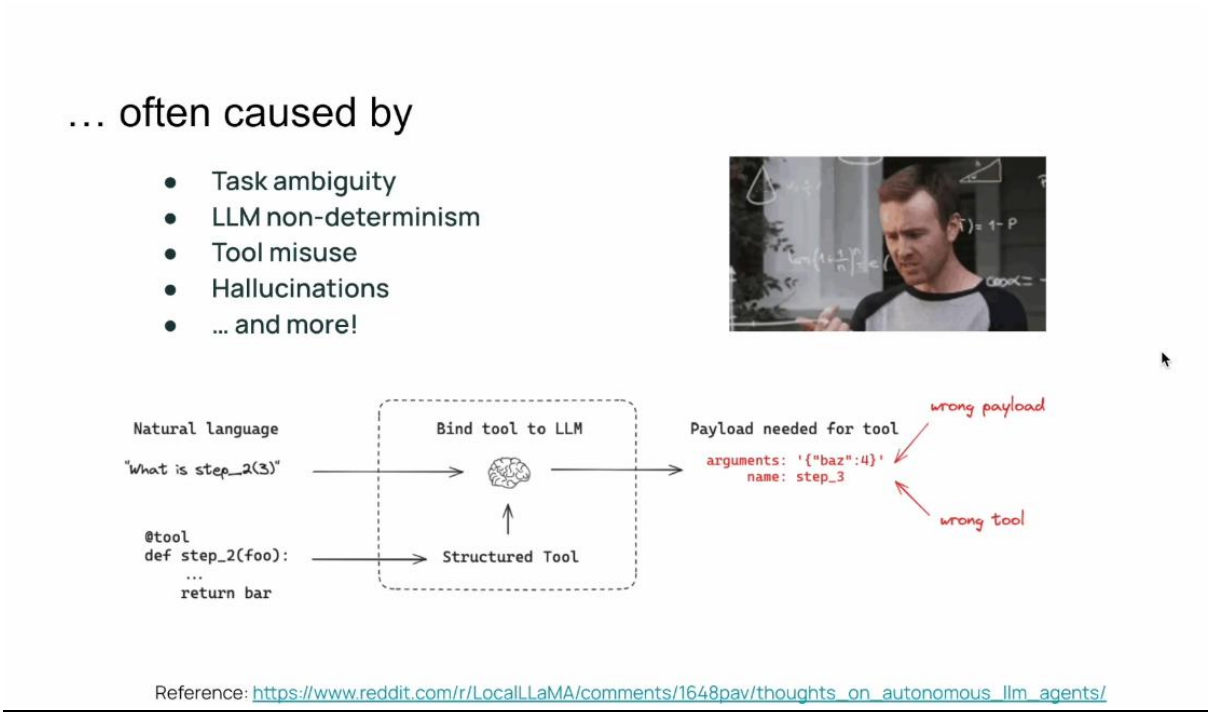
ReAct agents are flexible: any state possible!



Reference: https://react-lm.github.io/

… but ReAct agents suffer from poor reliability

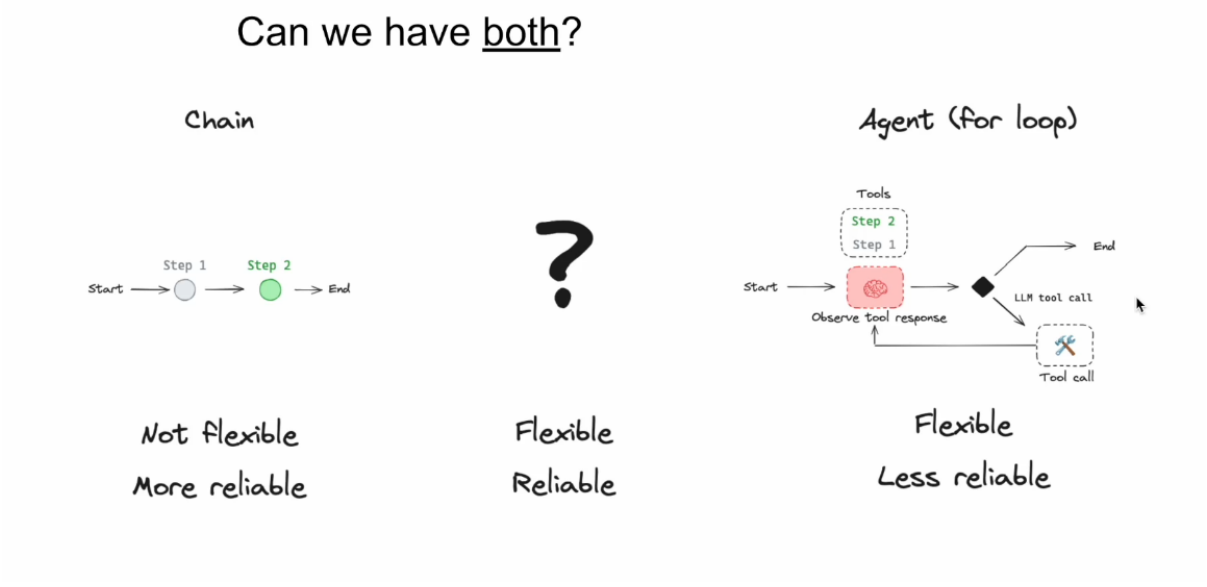

```
> Entering new AgentExecutor chain...
 I need to find out what time zone Paonia is in.
Action: Search
Action Input: "time zone Paonia"
Observation: The Current Time in. Paonia, Colorado is: Sunday 3/26/2023 2:48 PM MDT Paonia, Colorado is
Thought: I now know the time zone for Paonia
Action: Calculator
Action Input: 2:48 PM MDT
Observation: Answer: time data '2:48 PM MDT' does not match format '%I:%M %p %Z'
Thought: I need to convert the time to the correct format
Action: Calculator
Action Input: 2:48 PM MDT in Mountain Time Zone
Observation: Answer: type object 'datetime.datetime' has no attribute 'timezone'
Thought: I need to convert the time to the correct format
Action: Calculator
Action Input: 2:48 PM MDT in Mountain Time Zone to UTC
Observation: Answer: type object 'datetime.datetime' has no attribute 'timezone'
Thought: I need to convert the time to the correct format
Action: Calculator
Action Input: 2:48 PM MDT in Mountain Time Zone to UTC time
Observation: Answer: type object 'datetime.datetime' has no attribute 'timezone'
Thought: I need to convert the time to the correct format
Action: Calculator
Action Input: 2:48 PM MDT in Mountain Time Zone to UTC time format
Observation: Answer: type object 'datetime.datetime' has no attribute 'timezone'
Thought: I need to convert the time to the correct format
Action: Calculator
Action Input: convert 2:48 PM MDT in Mountain Time Zone to UTC time format
Observation: Answer: type object 'datetime.datetime' has no attribute 'timezone'
Thought: I need to convert the time to the correct format
Action: Calculator
Action Input: convert 2:48 PM MDT in Mountain Time Zone to UTC time format hh:mm
Observation: Answer: type object 'datetime.datetime' has no attribute 'timezone'
Thought: I need to convert the time to the correct format
Action: Calculator
Action Input: convert 2:48 PM MDT in Mountain Time Zone to UTC time format hh:mm:ss
Observation: Answer: type object 'datetime.datetime' has no attribute 'timezone'
Thought: I need to convert the time to the correct format
Action: Calculator
```

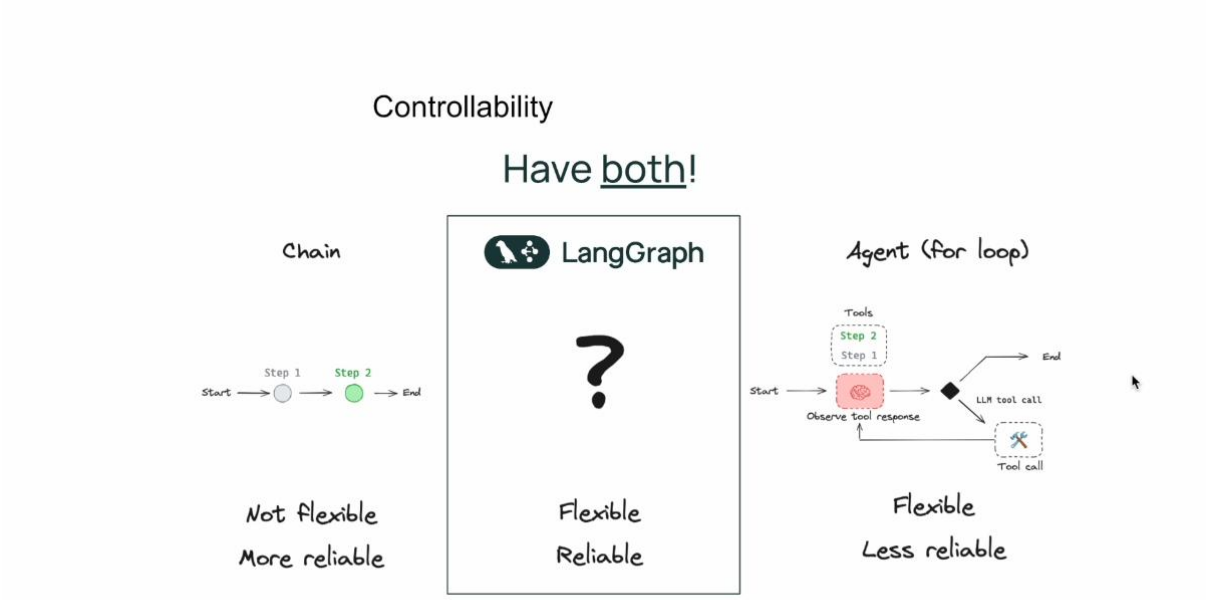


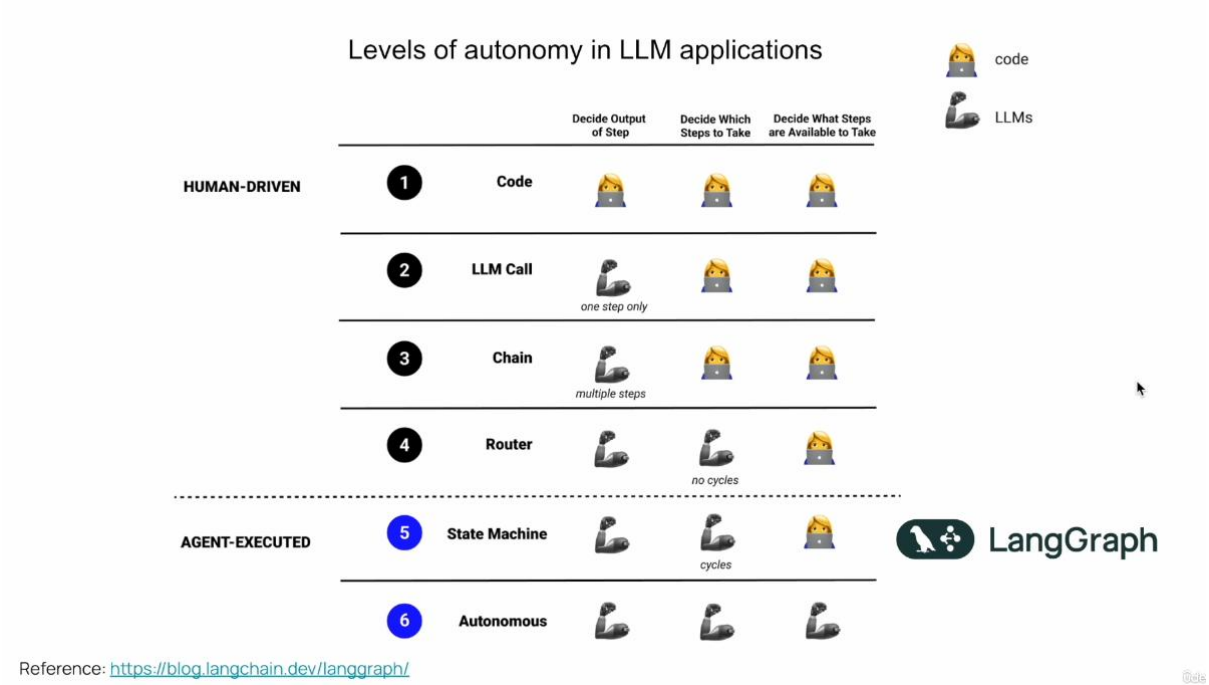
Level 5:

This is exactly why **LangGraph** was created, and you see this gap here between the autonomous agent and the router.

So here **LangGraph** is positioned, and the idea of **LangGraph** is to not give the entire freedom to the LLM, rather to scope it, and to take this freedom and take it down by one dimension, and to still allow the LLM to have freedom, but not to give it all the freedom.







Reference: https://blog.langchain.dev/langgraph/

And with **LangGraph**, we represent our software, our genetic software, as a graph with nodes and edges,

and we represent it as **a state machine** which can have cycles, which will give us a genetic properties,

and it will look like the agent can reason and it can think about what it needs to do.

And it gives us very advanced capabilities.

But we control the flow.

As developers, the LLM can play a crucial role in this flow and decide where do we need to go in this

flow.

But we as developers, we decide this flow.

And by actually reducing one dimension of freedom from the LLM, we can actually gain a lot of reliability,

and we can architect our system such that it would be much more resilient and reliable and all thanks,

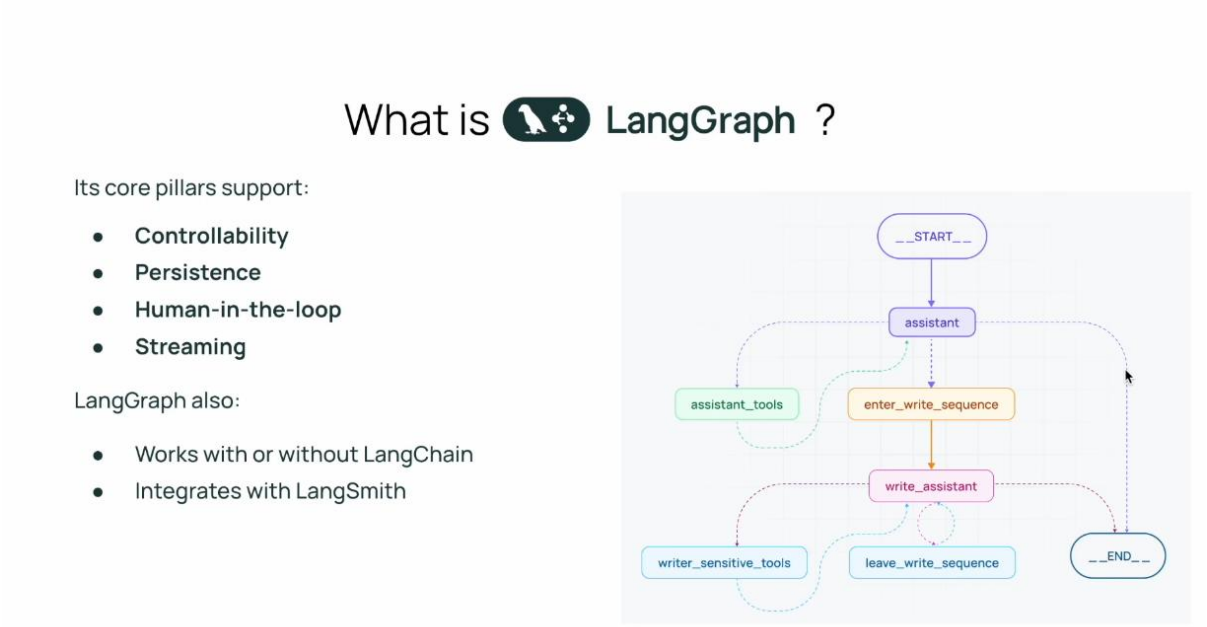
because we have the entire control of the flow.

And with **LangGraph**, **we architect our software, our genetic software as a state machine where we have**

nodes and edges, and this is displayed as a graph like you can see right here we have nodes and we

have edges and land graphs gives us a lot of support for building those kinds of graphs, which are

very opinionated for building a genetic applications.



And you might ask yourself, hey, why do we need **LangGraph** for that?

I can build it with airflow or with network X or another graph framework.

And the reason is that **LangGraph** is very opinionated towards genetic applications, and it's built to solve

that problem.

And it offers a lot of building blocks like controllability, running nodes in parallel and having conditional

branching with the LMS and having persistence, which is built in so we can store our current state

of the graph and what's being executed and what has been executed, which helps us to implement very

easily human in the loop flows where we integrate human feedback, which calibrates our execution of

our agent.

Time traveling, which is to replay some stuff that didn't work correctly, and even debugging and tooling

for tracing because it comes integrated with **LangGraph** out of the box.

And by the way, you can write inside graph any code you want so it doesn't have to be linked chain

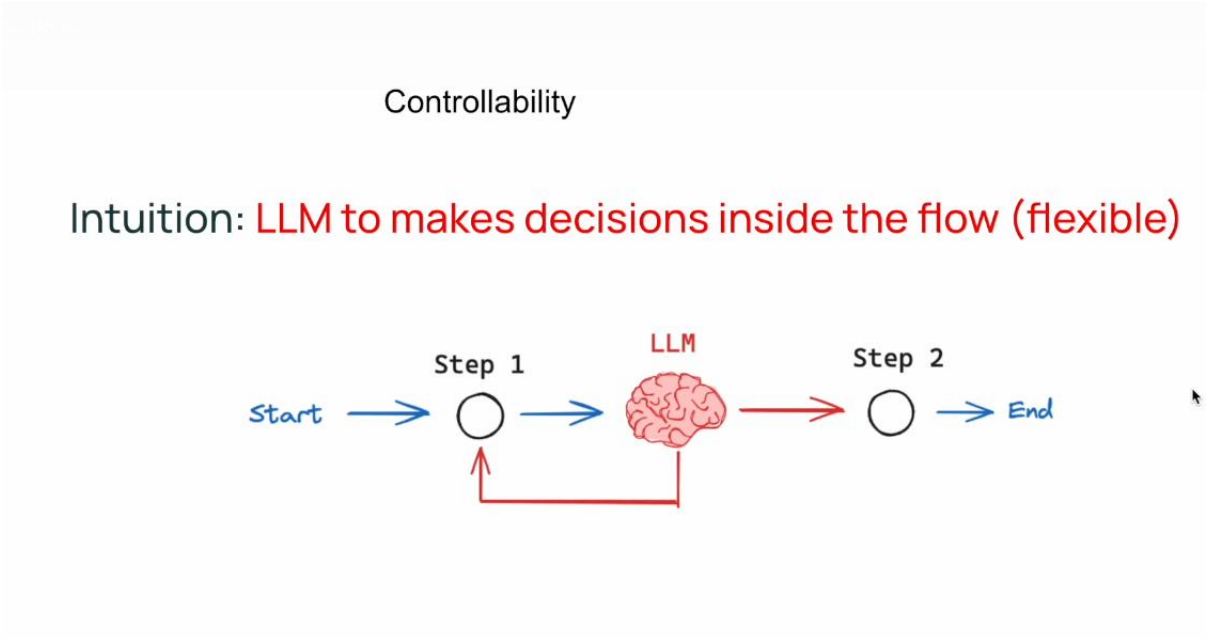
code, but you really can choose anything you want.

And by the way, I think one of the motivations for architecting these software as graphs is because

most of the papers on agentic applications and agentic behavior were illustrated as graphs, so it also

feels very natural to describe those solutions as graphs.

And it is very readable and and very easy to maintain and to test and to monitor.



So basically in LangGraph, we as developers, we control the flow and we write what is the flow?

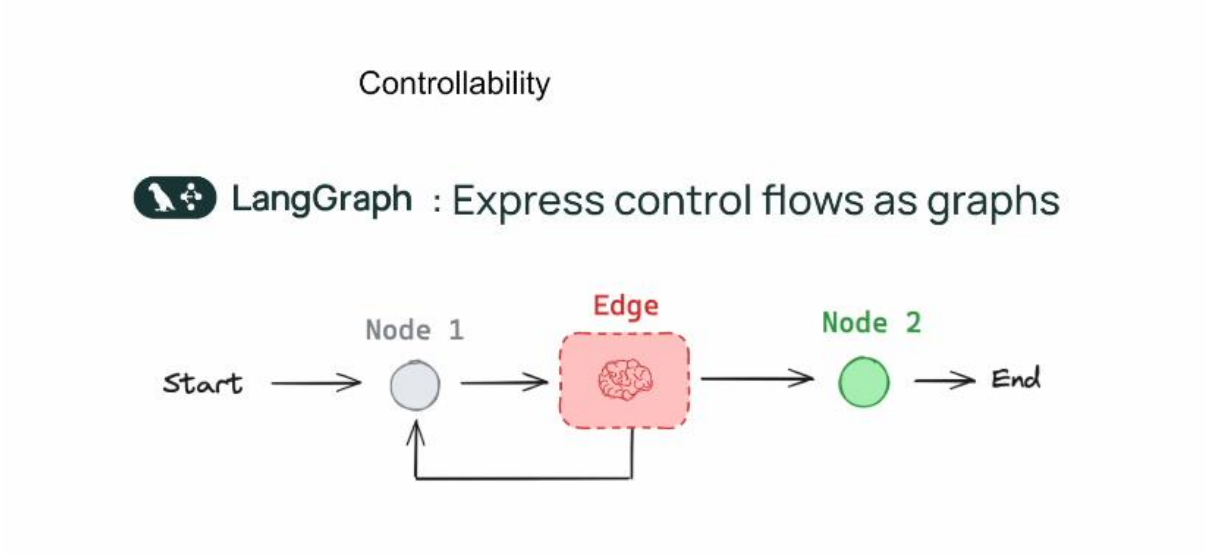
And we can integrate an LLM to decide where to go and what to execute in this flow.

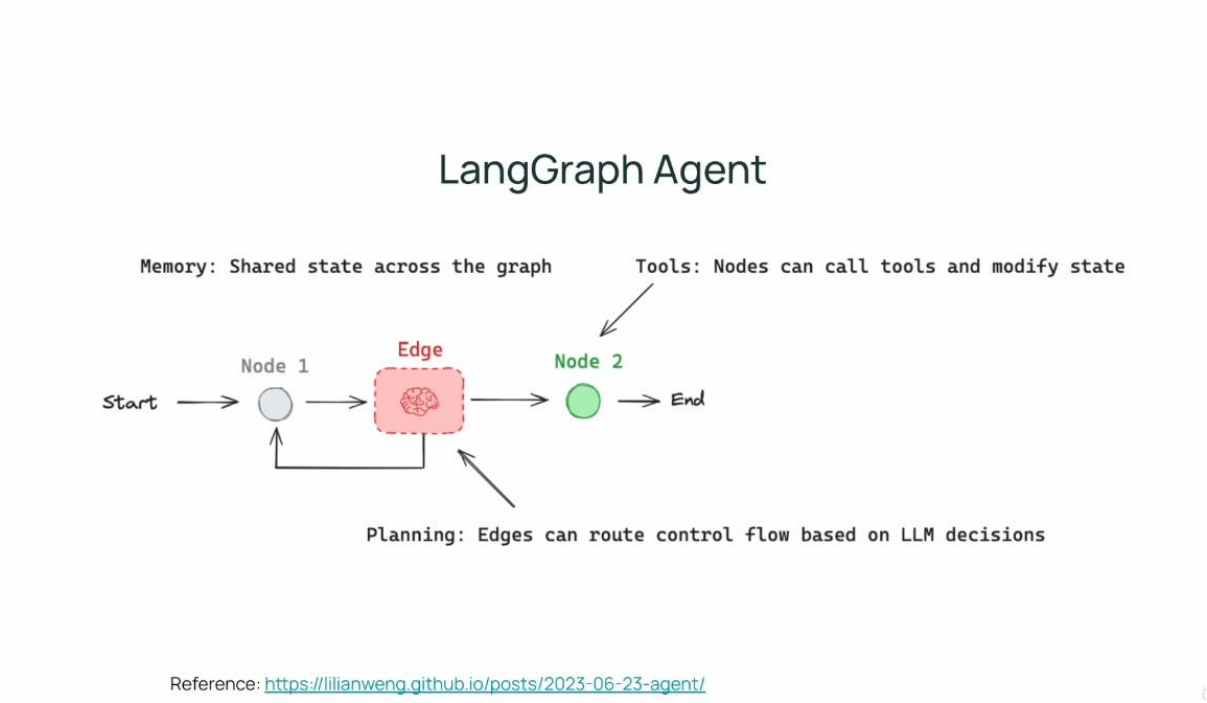
And with cycles this is important.

And we represent this flow with nodes and with edges.

So the LLM can use conditional branching to decide where to go and maybe to execute node one or node

two.





This is our state machine.

And because we have a state machine we need to have a state.

And the state is something which is shared across the nodes and across the edges.

And it's going to save all of our intermediate results, and it's going to give us useful information

into the LM, useful information to decide where to go in that flow.